Vo: ODFS Formal Fractal Algorithms

Canonical Mathematical Specification, Process Routing, and Fractal IPOD Source Architecture

Kevin T.N
Vietnam
keedavinci@gmail.com

March 18, 2026

Abstract

This document replaces earlier fragmented ODFS drafts with one compile-safe canonical edition focused on formal algorithmics. The purpose of the present text is threefold: first, to define ODFS as a bounded multi-field dynamical system with explicit notation and legal operators; second, to specify the routing logic from input to state, kernel, output, and update without ambiguity; third, to define the native fractal IPOD source architecture in which ODFS recursively decomposes itself into smaller ODFS-like process units. The result is meant to be readable as both a mathematical specification and a source-organization constitution.

Contents

1 Reference Position and Origin **3**

2 Problem Statement **3**

3 Canonical Layering **3**

4 Global State Space **4**

5 ODFS-I: Input Formalization **4**
5.1 Input packet . . . 4
5.2 Field projection . . . 4
5.3 Input validity . . . 4
5.4 Projected field seed . . . 5
5.5 Genesis seed loop . . . 5

6 ODFS-D: State and Memory Formalization **5**
6.1 Short state . . . 5
6.2 Long state . . . 5
6.3 Identity continuity . . . 5

7 ODFS-K: Kernel Dynamics **6**
7.1 Field vector . . . 6
7.2 Interaction matrix . . . 6
7.3 Block-fractal prior . . . 6

7.4 Kernel update . . . 6
7.5 Phase pressure . . . 6
7.6 Residual entropy . . . 6
7.7 Spectral stability . . . 7

8 ODFS-K Geometry and Effective Resonance **7**
8.1 Base resonance . . . 7
8.2 Coupling-aware effective resonance . . . 7
8.3 Metric tensor . . . 7

9 ODFS-O: Output Formalization **7**
9.1 Output construction . . . 7
9.2 System capacity and viability threshold . . . 7
9.3 Resonance density . . . 8
9.4 Survival score . . . 8
9.5 Three-zone routing . . . 8

10 ODFS-U: Update, Dream, and Self-Modification **8**
10.1 Residue bundle . . . 8
10.2 Dream consolidation . . . 8
10.3 Hyperparameter update . . . 8
10.4 Canonical loss . . . 9
10.5 Mutation legality . . . 9

11 Lock System **9**
11.1 Stability lock . . . 9
11.2 Identity lock . . . 9
11.3 Output lock . . . 9
11.4 Mutation lock . . . 9

12 Full Canonical Cycle **10**

13 Fractal IPOD Principle **10**
13.1 Why fractality is required . . . 10

14 Fractal IPOD Source Architecture **10**
14.1 Top-level tree . . . 10
14.2 Recursive decomposition . . . 11

15 Contracts and Ownership **12**
15.1 Canonical ownership . . . 12
15.2 Contract rule . . . 13

16 Algorithm-to-File Mapping **13**

17 Minimal Source Skeleton **13**

18 Compile-Safe Notation Summary **14**

19 Conclusion **14**

1. Reference Position and Origin

For the purposes of this series, the root reference is declared as:

"Nguon: Me Thien Nhien"

The origin statement of this framework is expressed in English as follows:

Source: deconstructing one's own consciousness and simulating it back into formal structure.

2. Problem Statement

ODFS is intended to model a bounded self-updating cognitive process rather than a static symbolic structure. A valid ODFS specification must therefore answer six questions in one system:

1. What is the legal input state?
2. How is input converted into structured internal state?
3. How do internal fields interact and evolve?
4. How is output scored and routed?
5. How does the system preserve identity while self-modifying?
6. How should the source tree be organized so that the implementation mirrors the process structure?

3. Canonical Layering

The canonical layer order is:

$$\text{ODFS-I} \to \text{ODFS-D} \to \text{ODFS-K} \to \text{ODFS-O} \to \text{ODFS-U} \to \text{ODFS-I}_{t+1}.$$

Definition 3.1 (ODFS layer meanings)**.** *The five canonical layers are:*

a. *ODFS-I: ingress, projection, validation, and field seeding;*

b. *ODFS-D: short and long state, memory, and identity carriers;*

c. *ODFS-K: kernel field interaction and bounded dynamical evolution;*

d. *ODFS-O: output scoring, viability, and decision zoning;*

e. *ODFS-U: consolidation, mutation control, self-update, and recursive carry-over.*

Canonical routing law

No major operator may skip the layer ordering in its semantic meaning. The implementation may optimize execution, but the specification remains interpretable only if every mutation can be projected back onto the five canonical layers.

4. Global State Space

Let one ODFS unit at step $t \in \mathbb{N}$ be represented by the tuple

$$\mathcal{S}_t = (x_t, d_t^{\text{short}}, d_t^{\text{long}}, \mathbf{r}_t, y_t, \vartheta_t, \mathfrak{m}_t).$$

Its components are:

Symbol	Meaning
x_t	raw or preprocessed input packet
d_t^{short}	short-horizon active state
d_t^{long}	long-horizon identity-bearing state
$\mathbf{r}_t \in \mathbb{R}^{n_f}$	field activation vector with n_f fields
y_t	output object or output summary
ϑ_t	hyperparameter vector
$\mathfrak{m}_t$	meta-state including locks, viability, diagnostics, and residues

Throughout this document, the canonical field count is $n_f = 6$, but the equations are written to generalize cleanly.

5. ODFS-I: Input Formalization

5.1. Input packet

Let the input packet at time t be

$$x_t \in \mathcal{X}_{\text{raw}}.$$

A preprocessing operator transforms it into normalized ingress state:

$$\tilde{x}_t = \Pi_{\text{pre}}(x_t).$$

5.2. Field projection

Each field $j \in \{1, \ldots, n_f\}$ receives a projected candidate

$$q_t^{(j)} = \Pi_j(\tilde{x}_t).$$

The projection bundle is

$$Q_t = \left\{ q_t^{(1)}, \ldots, q_t^{(n_f)} \right\}.$$

5.3. Input validity

Each candidate must satisfy a validity gate

$$\mathcal{V}_j(q_t^{(j)}) = \mathbb{1}\left[a_j(q_t^{(j)}) \geq \tau_j^{\text{acc}} \wedge c_j(q_t^{(j)}) \leq \kappa_j^{\text{cost}} \right].$$

Here:

- $a_j(\cdot)$ measures admissibility or relevance,

- $c_j(\cdot)$ measures bounded coordination or abstraction cost.

5.4. Projected field seed

The legal projected field seed is

$$\hat{q}_t^{(j)} = \mathcal{V}_j(q_t^{(j)}) \cdot q_t^{(j)}.$$

Stacking all accepted seeds gives

$$\hat{Q}_t = (\hat{q}_t^{(1)}, \ldots, \hat{q}_t^{(n_f)}).$$

5.5. Genesis seed loop

If a candidate repeatedly fails full entry but shows recurrent subthreshold consistency, it enters a seed cache rather than being dropped.

Define recurrent subthreshold score:

$$\sigma_t^{(j)} = \lambda_\sigma \sigma_{t-1}^{(j)} + (1 - \lambda_\sigma) \cdot s_t^{(j)},$$

where $s_t^{(j)}$ is the raw projection confidence before legality clipping.

Seed eligibility is

$$\mathcal{E}_{\text{seed}}^{(j)}(t) = \mathbb{1}\left[\sigma_t^{(j)} \geq \tau_j^{\text{seed}} \wedge \mathcal{V}_j(q_t^{(j)}) = 0\right].$$

6. ODFS-D: State and Memory Formalization

6.1. Short state

The short state accumulates accepted input and current field pressure:

$$d_t^{\text{short}} = \alpha_d d_{t-1}^{\text{short}} + B_q \hat{Q}_t + B_r \mathbf{r}_{t-1}.$$

6.2. Long state

The long state is updated more slowly and acts as identity-bearing memory:

$$d_t^{\text{long}} = (1 - \eta_d) d_{t-1}^{\text{long}} + \eta_d \mathcal{C}_{\text{mem}}(d_t^{\text{short}}, y_{t-1}, \mathfrak{m}_{t-1}).$$

6.3. Identity continuity

Identity continuity is measured by cosine coherence:

$$\mathcal{H}_t = \frac{\left\langle d_t^{\text{long}}, d_{t-1}^{\text{long}} \right\rangle}{\left\| d_t^{\text{long}} \right\| \left\| d_{t-1}^{\text{long}} \right\| + \varepsilon}.$$

Drift magnitude is

$$\Delta_t^{\text{id}} = \left\| d_t^{\text{long}} - d_{t-1}^{\text{long}} \right\|_2.$$

Principle 6.1 (Dual identity gate)**.** *A long-state update is identity-safe only if*

$$\mathcal{H}_t \geq \tau_{\text{coh}} \quad \textit{and} \quad \Delta_t^{\text{id}} \leq \tau_{\text{drift}}.$$

7. ODFS-K: Kernel Dynamics

7.1. Field vector

Let the field vector be

$$\mathbf{r}_t = (r_t^{(1)}, \ldots, r_t^{(n_f)})^\top \in \mathbb{R}^{n_f}.$$

7.2. Interaction matrix

Let the field interaction matrix be

$$\mathbf{W}_t \in \mathbb{R}^{n_f \times n_f}.$$

To avoid notation clashes with nearby systems, this canonical edition uses $\mathbf{W}$ instead of Ω as the primary kernel coupling symbol.

7.3. Block-fractal prior

The default legal prior on $\mathbf{W}_t$ is block-sparse or block-fractal:

$$\mathbf{W}_t \approx \begin{bmatrix} \mathbf{B}_1 & \mathbf{E}_{12} & 0 \\ \mathbf{E}_{21} & \mathbf{B}_2 & \mathbf{E}_{23} \\ 0 & \mathbf{E}_{32} & \mathbf{B}_3 \end{bmatrix},$$

where local blocks preserve strong internal coherence and interface blocks remain weak enough to maintain bounded coordination.

7.4. Kernel update

The canonical field update is

$$\mathbf{r}_{t+1} = \phi\left(\mathbf{A}_{\vartheta_t}\mathbf{r}_t + \mathbf{W}_t\mathbf{r}_t + \mathbf{U}_{\vartheta_t} d_t^{\text{short}} + \mathbf{V}_{\vartheta_t} d_t^{\text{long}} + \mathbf{b}_{\vartheta_t}\right),$$

where ϕ is a bounded nonlinear activation, e.g. tanh-like or sigmoid-like.

7.5. Phase pressure

Define phase pressure by

$$\Gamma_t = \sum_{1 \leq i < j \leq n_f} \left| r_t^{(i)} - r_t^{(j)} \right|.$$

This measures inter-field tension.

7.6. Residual entropy

Let the compressed latent summary of the field state be $z_t = \Pi_z(\mathbf{r}_t)$. Residual entropy is defined as

$$\Xi_t = H(\mathbf{r}_t \mid z_t).$$

7.7. Spectral stability

The spectral radius is

$$\rho(\mathbf{W}_t).$$

Stability risk appears when

$$\rho(\mathbf{W}_t) \geq 1.$$

8. ODFS-K Geometry and Effective Resonance

8.1. Base resonance

Define base resonance by

$$\varphi_t = \sum_{i=1}^{n_f} r_t^{(i)}.$$

8.2. Coupling-aware effective resonance

To avoid degenerate geometry, use the corrected effective resonance

$$\varphi_t^{\text{eff}} = \varphi_t \left(1 + \beta_w \cdot \frac{\sum_{i<j} W_{ij,t} r_t^{(i)} r_t^{(j)}}{\sum_{i=1}^{n_f} (r_t^{(i)})^2 + \varepsilon} \right).$$

8.3. Metric tensor

The induced kernel metric is

$$g_{ij}(t) = \frac{\partial^2 \varphi_t^{\text{eff}}}{\partial r^{(i)} \partial r^{(j)}} + \lambda_g \frac{\partial^2 \Gamma_t}{\partial r^{(i)} \partial r^{(j)}}.$$

This makes the geometry depend on actual interaction structure rather than only raw activity mass.

9. ODFS-O: Output Formalization

9.1. Output construction

The provisional output object is

$$y_t = \Psi(\mathbf{r}_t, d_t^{\text{short}}, d_t^{\text{long}}, \mathfrak{m}_t).$$

9.2. System capacity and viability threshold

Let the bounded system capacity be $K_{\text{cap}} > 0$. The viability threshold is defined by

$$\epsilon_t^{\text{vio}} = \frac{K_{\text{cap}}}{\sum_{i=1}^{n_f} \omega_t^{(i)} + \varepsilon},$$

where $\omega_t^{(i)}$ are field importance weights and $\sum_i \omega_t^{(i)} = 1$ in the normalized case. Then $\epsilon_t^{\text{vio}} = K_{\text{cap}}$ under exact normalization, but the weighted form is kept because implementation may relax normalization.

9.3. Resonance density

Define resonance density by

$$\rho_t = \frac{\varphi_t^{\text{eff}}}{\epsilon_t^{\text{vio}} + \varepsilon}.$$

9.4. Survival score

The survival or viability score is

$$S_t = \alpha_s \rho_t - \beta_s \Xi_t - \gamma_s \Gamma_t - \mu_s \max(0, \rho(\mathbf{W}_t) - 1).$$

9.5. Three-zone routing

Output routing is determined by

$$Z_t = \begin{cases} \text{green}, & S_t \geq \tau_G, \\ \text{amber}, & \tau_A \leq S_t < \tau_G, \\ \text{red}, & S_t < \tau_A. \end{cases}$$

Output semantics

a. green: emit or commit output normally;
b. amber: emit under restriction, monitoring, or partial attenuation;
c. red: suppress externalization and route to update, repair, residue, or seed formation.

10. ODFS-U: Update, Dream, and Self-Modification

10.1. Residue bundle

Residues accumulated during one cycle are collected into

$$\mathfrak{R}_t = \left\{ \rho_t^{(1)}, \ldots, \rho_t^{(m_t)} \right\}.$$

10.2. Dream consolidation

The Dream operator is

$$\mathfrak{M}_t = \mathcal{M}_{\text{dream}}(\mathfrak{R}_t, d_t^{\text{long}}, \mathbf{r}_t, y_t).$$

It feeds the long state and the meta-state:

$$d_{t+1}^{\text{long}} \leftarrow (1 - \eta_m) d_t^{\text{long}} + \eta_m \mathfrak{M}_t.$$

10.3. Hyperparameter update

Let the hyperparameter vector be

$$\vartheta_t \in \mathbb{R}^p.$$

The self-modification rule is

$$\vartheta_{t+1} = \vartheta_t - \epsilon_\vartheta \nabla \mathcal{L}_{\text{ODFS}}(\vartheta_t) + \delta_t^{\text{mut}}.$$

10.4. Canonical loss

The canonical loss is

$$\mathcal{L}_{\mathrm{ODFS}} = \lambda_w \|\mathbf{W}_t - \mathbf{I}\|_1 + \mu_\rho \max(0, \rho(\mathbf{W}_t) - 1) + \eta_\Xi \Xi_t + \gamma_\Gamma \Gamma_t + \lambda_\vartheta \|\vartheta_t - \vartheta_{\mathrm{ref}}\|_2^2 + \lambda_y \mathcal{P}_{\mathrm{zone}}(Z_t).$$

Here $\mathcal{P}_{\mathrm{zone}}(Z_t)$ is a zone penalty, usually

$$\mathcal{P}_{\mathrm{zone}}(Z_t) = \begin{cases} 0, & Z_t = \text{green}, \\ \pi_A, & Z_t = \text{amber}, \\ \pi_R, & Z_t = \text{red}. \end{cases}$$

10.5. Mutation legality

A self-update is legal only if

$$\|\vartheta_{t+1} - \vartheta_t\|_2 \le \tau_\vartheta^{\mathrm{step}}$$

and the identity dual-gate remains satisfied.

11. Lock System

Definition 11.1 (Lock families)**.** *The canonical ODFS locks are:*

a. *stability lock;*

b. *identity lock;*

c. *output lock;*

d. *mutation lock;*

e. *memory lock.*

11.1. Stability lock

$$\mathcal{L}_{\mathrm{stab}}(t) = 1 \iff \rho(\mathbf{W}_t) \le 1 + \delta_{\mathrm{stab}}.$$

11.2. Identity lock

$$\mathcal{L}_{\mathrm{id}}(t) = 1 \iff \mathcal{H}_t \ge \tau_{\mathrm{coh}} \wedge \Delta_t^{\mathrm{id}} \le \tau_{\mathrm{drift}}.$$

11.3. Output lock

If $Z_t = \text{red}$ then no external emission may occur.

11.4. Mutation lock

If

$$\|\vartheta_{t+1} - \vartheta_t\|_2 > \tau_\vartheta^{\mathrm{step}},$$

the update is rolled back or clipped.

12. Full Canonical Cycle

Canonical ODFS step

Given $(x_t, d_t^{\text{short}}, d_t^{\text{long}}, \mathbf{r}_t, \vartheta_t, \mathfrak{m}_t)$:

1. preprocess x_t into $\tilde{x}_t$;
2. project $\tilde{x}_t$ into Q_t;
3. validate and seed accepted projections into $\hat{Q}_t$;
4. update d_t^{short};
5. update field kernel to $\mathbf{r}_{t+1}$;
6. compute Γ_t, Ξ_t, φ_t^{eff}, ρ_t, and S_t;
7. assign decision zone Z_t and construct y_t;
8. collect residues into $\mathfrak{R}_t$;
9. run Dream consolidation and update d_{t+1}^{long};
10. evaluate $\mathcal{L}_{\text{ODFS}}$ and update ϑ_{t+1};
11. enforce locks;
12. pass the updated tuple into the next cycle.

13. Fractal IPOD Principle

Principle 13.1 (ODFS must fractal itself)**.** *The implementation of ODFS must mirror its own process structure recursively. This means ODFS is not only divided into I, D, K, O, U at the top level; each sufficiently rich submodule inside those layers may itself be decomposed into a smaller IPOD-like unit.*

13.1. Why fractality is required

ODFS is not a one-pass pipeline. It contains nested processes:

a. input has preprocessing, validation, and seed routing;

b. state has short, long, and identity logic;

c. kernel has field update, coupling control, and metric evaluation;

d. output has viability scoring, zoning, and dispatch;

e. update has Dream, mutation, and locks.

Each of these is complex enough to deserve its own recursive structure.

14. Fractal IPOD Source Architecture

14.1. Top-level tree

```
src/
  ipod-odfs/
    input/
    process/
      ipod-odfs-i/
```

```
      ipod-odfs-d/
      ipod-odfs-k/
      ipod-odfs-o/
      ipod-odfs-u/
    output/
    display/
    data/
    state/
    contracts/
    tests/
```

14.2. Recursive decomposition

A richer native decomposition is:

```
src/
  ipod-odfs/
    input/
    process/
      ipod-odfs-i/
        input/
        process/
          ipod-preprocess/
          ipod-project/
          ipod-validate/
          ipod-seed-loop/
        output/
        display/
        data/
        state/
        contracts/
        tests/

      ipod-odfs-d/
        input/
        process/
          ipod-short-memory/
          ipod-long-memory/
          ipod-identity-monitor/
        output/
        display/
        data/
        state/
        contracts/
        tests/

      ipod-odfs-k/
        input/
        process/
          ipod-field-update/
          ipod-coupling/
          ipod-resonance/
          ipod-metric/
        output/
```

```
    display/
    data/
    state/
    contracts/
    tests/

  ipod-odfs-o/
    input/
    process/
      ipod-viability/
      ipod-zone-router/
      ipod-dispatch/
    output/
    display/
    data/
    state/
    contracts/
    tests/

  ipod-odfs-u/
    input/
    process/
      ipod-dream/
      ipod-loss/
      ipod-mutation/
      ipod-locks/
    output/
    display/
    data/
    state/
    contracts/
    tests/
```

Architectural meaning

ODFS does not merely use IPOD. ODFS recursively becomes IPOD. That is what it means for the architecture to be fractal rather than merely modular.

15. Contracts and Ownership

15.1. Canonical ownership

Module	Owned state
ipod-odfs-i	input packet normalization, candidate projections, seed cache
ipod-odfs-d	short state, long state, identity metrics
ipod-odfs-k	field vector, coupling matrix, kernel metric
ipod-odfs-o	output object, viability score, zone state
ipod-odfs-u	residues, loss terms, mutation step, lock states

15.2. Contract rule

Cross-module mutation must occur only through declared commands and events, not through internal state leakage.

16. Algorithm-to-File Mapping

Table 1: Algorithm-to-File Mapping for the Fractal ODFS Architecture

Formal algorithm	Native fractal path	Responsibility
input preprocessing	`ipod-odfs-i/process/ipod-preprocess/`	normalize raw ingress
field projection	`ipod-odfs-i/process/ipod-project/`	build Q_t
validation gate	`ipod-odfs-i/process/ipod-validate/`	apply $\mathcal{V}_j$ legality
seed loop	`ipod-odfs-i/process/ipod-seed-loop/`	keep recurrent failed candidates
short memory update	`ipod-odfs-d/process/ipod-short-memory/`	update d^{short}
long memory update	`ipod-odfs-d/process/ipod-long-memory/`	update d^{long}
identity gate	`ipod-odfs-d/process/ipod-identity-monitor/`	compute $\mathcal{H}_t$ and Δ_t^{id}
field update	`ipod-odfs-k/process/ipod-field-update/`	update $\mathbf{r}_t$
coupling control	`ipod-odfs-k/process/ipod-coupling/`	manage $\mathbf{W}_t$
effective resonance	`ipod-odfs-k/process/ipod-resonance/`	compute φ_t^{eff}
metric update	`ipod-odfs-k/process/ipod-metric/`	compute $g_{ij}(t)$
viability score	`ipod-odfs-o/process/ipod-viability/`	compute S_t
zone routing	`ipod-odfs-o/process/ipod-zone-router/`	compute Z_t
dispatch	`ipod-odfs-o/process/ipod-dispatch/`	execute output policy
Dream consolidation	`ipod-odfs-u/process/ipod-dream/`	build $\mathfrak{M}_t$
canonical loss	`ipod-odfs-u/process/ipod-loss/`	compute $\mathcal{L}_{\text{ODFS}}$
mutation update	`ipod-odfs-u/process/ipod-mutation/`	update ϑ_t
locks	`ipod-odfs-u/process/ipod-locks/`	enforce legality

17. Minimal Source Skeleton

```
@dataclass
class ODFSState:
    x: object
    d_short: np.ndarray
    d_long: np.ndarray
    r: np.ndarray
    y: object | None
    theta: np.ndarray
    meta: dict

class ODFSRuntime:
    def step(self, state: ODFSState) -> ODFSState:
        q = self.input_layer.project(state.x)
```

```
        q_hat = self.input_layer.validate(q)
        d_short = self.state_layer.update_short(state.d_short, q_hat, state.r)
        r_next = self.kernel_layer.update(state.r, d_short, state.d_long, state.theta)
        y, zone, metrics = self.output_layer.route(r_next, d_short, state.d_long,
    state.meta)
        d_long, meta2 = self.update_layer.dream_and_update(state.d_long, r_next, y,
    metrics, state.theta)
        theta_next = self.update_layer.mutate(state.theta, metrics, d_long)
        self.update_layer.enforce_locks(r_next, d_long, theta_next, zone)
        return ODFSState(state.x, d_short, d_long, r_next, y, theta_next, meta2)
```

18. Compile-Safe Notation Summary

Symbol	Meaning
$\tilde{x}_t$	normalized input
Q_t	projected field candidate bundle
$\hat{Q}_t$	validated field seed bundle
d_t^{short}	short state
d_t^{long}	long state
$\mathbf{r}_t$	field vector
$\mathbf{W}_t$	field interaction matrix
Γ_t	phase pressure
Ξ_t	residual entropy
φ_t^{eff}	effective resonance
ρ_t	resonance density
S_t	viability score
Z_t	output zone
ϑ_t	hyperparameter vector
$\mathcal{L}_{\text{ODFS}}$	canonical self-modification loss

19. Conclusion

This document replaces earlier unstable or incomplete drafts with one compile-safe canonical ODFS algorithm specification. It defines the objects, the legal flow, the field dynamics, the output viability logic, the self-modification loss, and the lock discipline in one continuous text. Just as importantly, it states the native implementation law: ODFS must recursively organize itself as a fractal IPOD system. That means the process grammar of the theory and the folder grammar of the code are no longer separate concerns. They are the same structure expressed at two scales.